

Nombre: <u>Gabriel Gómez</u>	Código:	e-mail: <u>g.gomez1@uniand.edu.co</u>
------------------------------	---------	---------------------------------------

We are set to build open classes. An open class is one to which new methods can be added without editing the class directly.

Object definition:

The first step to open classes is to define the way in which objects can be extended. To make this easier, we will focus on the extension of objects' behavior only.

To extend objects we will split methods in two groups "provided" and "required" methods. Provided methods use the `provided` keyword before the method definition and are fully implemented. Required methods use the `required` keyword before the method definition, and are only defined by the method signature (selector and parameters) followed by the `end` keyword.

Task 1. [0.7/5] Implement a `comparable` object with no attributes, that has two required methods (`<`, `==`) and four provided methods (`<=`, `>`, `>=`, `!=`) (you do not need to provide the implementations for the methods, just the method definitions and add ... as the body. The methods should receive two parameters, the two objects to compare.

Fun { Comparable }

Local

providedMethods = { NewCell nil }

requiredMethods = { NewCell nil }

fun { LessThan Object1 Object2 }

⋮ R = LessThan (keyword: required met: LessThan)

end

Fun { Equal Object1 Object2 }

⋮ R = Equal (keyword: required met: Equal)

end

Fun { LessThanOrEqual Object1 Object2 }

⋮ R = LessThan (keyword: provided met: LessThanOrEqual)

end

Fun { GreaterThan Object1 Object2 }

⋮ R = GreaterThan (keyword: provided met: GreaterThan)

end

Fun {

Fun { GreaterThanOrEqual }

⋮ R = GreaterThan (keyword: provided met: GreaterThanOrEqual)

end

Fun { Different }

⋮ R = Different (keyword: provided met: Different)

end

in

comparable (privateMethods: [LessThan, Equal] requiredMethods: [LessThanOrEqual

GreaterThan

GreaterThanOrEqual

Different])

end

Task 2. [1.8/5] The purpose of open classes is that their methods can change their implementation at run time. In particular, required methods would need an implementation for the objects to work. As a consequence we need to modify the Mozart definition of methods, so that they can change. How would you define methods so that they can be modified?

1. [1.4/5] Provide the implementation of methods so that they can change (following the functional representation of objects used in the project). You can sketch the implementation using the comparable object from Task 1.

2. [0.4/5] Sketch the storage strategy for methods (in terms of scope/stores/memory/semantics or other elements that you consider relevant).

① **Fun { Change Methods : Object Method }**

RequireMethods = { Condselect Object requireMethods nil }

for Meth in RequireMethods do

Name = { Label Meth }

if Name == Method then

NewImpl = fun { } = Soy su nueva implementación end }

Keyword = provided % si quiere cambiar de keyword.

{ Record.adjoin At Meth keyword Keyword } % Actualiza

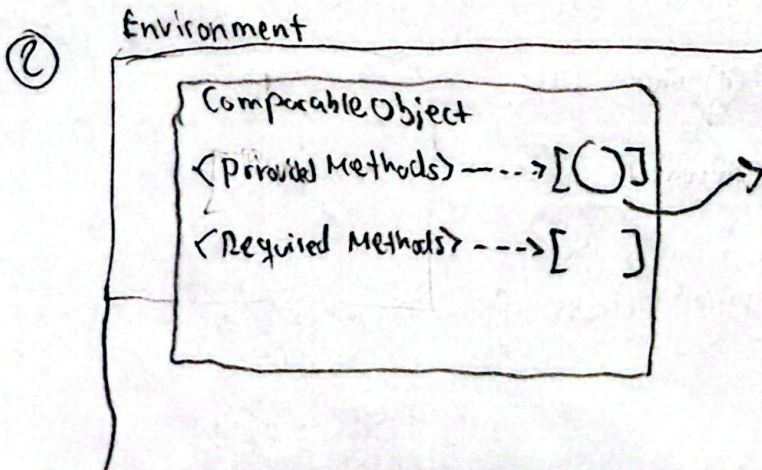
{ Record.adjoin At Meth met NewImpl } % Actualiza

end

end

end

Solo con records
no funciona



To give implementations for required methods we need to define a new operation.

Task 3. [1.9/5] Implement a `extendedBy` function that given an object definition and any number of objects, adds the functions implemented in the set of objects to the first object. For the implementation of this function you may assume that other functions you need are implemented, but you need to state the specification of additional functions used.

0.2 1. [0.2/5] Give an example of the syntax of the `extendedBy` function

1.5 2. [1.6/5] Give an implementation of the `extendedBy` function. Only required methods should be replaced, and once a required method is fulfilled, its implementation cannot be replaced again.

0.1 3. [0.1/5] Is `extendedBy` a meta-function?

$Obj = \{NewObject\}$

① $Comp1 = \{Comparable\}$

$Comp2 = \{Comparable\}$

$Extended = \{ExtendedBy\} Obj [Comp1 Comp2]\}$

② $fun \{ ExtendedBy \} Object ObjectsList \}$

for Obj in $ObjectsList$ do

$functions = \{ CondSelect \} Obj \{ requiredMethods \} nil \}$

for $Function$ in $functions$ do

if $\{ Condselect \} Function \{ met \} nil \} \neq nil$ then $\% Esto valida que este sin implementar (fulfilled)$

$List = \{ CondSelect \} Object \{ methods \} nil \}$

$updatedList = List \uparrow Function \% agrega la funcion, actualiza object$

$Object := \{ Record \cdot adjoinAt \} Object \{ methods \} updatedList \}$

end
end
end
 $\rightarrow$ realmente no hace la modificación

③ Sí lo es, esta cambiando la estructura a un objeto, le esta agregando metodos!

Task 4. [0.6/5] An interval is an object with a **start**, a **finish**, a **size** and the behavior to determine if an interval contains an element or not. Additionally an interval knows how to go inspect **forEach** of its elements.

An **enumerable** is an object that should be able to iterate over a family of component objects (**forEach**) although the implementation of such behavior is specific per object type. Additionally, enumerables have the behavior to **map**, **filter**, and **reduce** their elements.

0.4 1. [0.5/5] Create a comparable and enumerable interval

2. [0.1/5] Is the resulting object complete (is fully implemented)?

```
fun {Interval InitStart InitFinish InitSize}
```

```
  start = {NewCell InitStart} Interval = {NewCell interval()}
```

```
  finish = {NewCell InitFinish}
```

```
  size = {NewCell InitSize}
```

```
  %Obtiene todos los metodos del comparable
```

```
  Comparable = {Comparable}
```

```
  for Meth in {CondSelect Comparable requiredMethods nil} do
```

```
    List = {Condselect @Interval methods nil}
```

```
    Interval := {Record.adjoinAt @Interval methods List | Meth}
```

```
  end
```

```
  %Obtiene todos los metodos del enumerable.
```

```
  Enumerable = {Enumerable}
```

```
  Rec = {ExtendedBy @Interval Enumerable}
```

```
  end
```

! comparable?

```
end
```

```
end
```

Task 5. Extend the extendedBy function so that:

- 1.4 1. You can extend provided and required methods of a function
- 0.1 2. The extended function receives a linearization function (that you need to implement) stating how, and if methods of the first object are fulfilled by methods of the other objects.

① Agregar

% falta un for para los provided

functions = { CondSelect Obj providedMethods nil }

for Function in functions do

List = { Condselect Object methods nil }

UpdatedList = List | Function

Object := { Record.adjoinAt object methods UpdatedList }

end

② fun { Linearization } *No extra linearizando las multiples implementaciones*
if { HasFeature Object Message } == False then

for (Obj in ObjectsList do

if { HasFeature Obj Message } == true then

canRespond := True

Value = { Obj . Message }

end

end

if @canRespond == true

Value

else = No entendio el mensaje